

Logical Reasoning for Planning

Warehouse Robot Planning Agent

Name: Sidh Virmani

Course: Artificial Intelligence

GitHub: https://github.com/Sidh-Virmani/AI_Logic_Lab

1 Planning Problem

The planning problem is represented as

$$(I, A, G)$$

where I is the initial state, A is the set of actions, and G is the goal.

$$I = \{At(Robot, A), At(Package, A)\}$$

$$G = \{At(Package, C)\}$$

The warehouse connections are

$$A \leftrightarrow B \leftrightarrow C.$$

The main actions are:

Action	Preconditions	Effects
$Move(X, Y)$	$At(Robot, X)$	$\neg At(Robot, X), At(Robot, Y)$
$PickUp(Package, L)$	$At(Robot, L), At(Package, L)$	$\neg At(Package, L), Holding(Package)$
$Drop(Package, L)$	$At(Robot, L), Holding(Package)$	$\neg Holding(Package), At(Package, L)$

Initially, $PickUp(Package, A)$ is applicable because both required facts are true. $Drop(Package, C)$ is not applicable because the robot is not at C and is not holding the package.

2 Manual Plan

A valid plan is

$$\boxed{PickUp(Package, A) \rightarrow Move(A, B) \rightarrow Move(B, C) \rightarrow Drop(Package, C)}$$

The states are:

$$S_0 = \{At(Robot, A), At(Package, A)\}$$

$$S_1 = \{At(Robot, A), Holding(Package)\}$$

$$S_2 = \{At(Robot, B), Holding(Package)\}$$

$$S_3 = \{At(Robot, C), Holding(Package)\}$$

$$S_4 = \{At(Robot, C), At(Package, C)\}$$

Since $At(Package, C) \in S_4$, the goal is satisfied.

3 Prompt Used

Implement the warehouse logical planning problem from the attached lab sheet in Python. Represent states as sets of propositions and actions using positive/negative preconditions and effects. Use BFS to find a valid plan, report when no plan exists, and print the action sequence and states reached. Include Tests A, B, and C and detailed explanatory comments.

The exact prompt is stored in `prompt.txt`, and the implementation is stored in `Code.py`.

4 Implementation

Each action stores:

- positive and negative preconditions;
- positive and negative effects.

An action is applicable only if its positive preconditions are present and its negative preconditions are absent.

The successor state is

$$S' = (S - Effects^-) \cup Effects^+.$$

Breadth-First Search explores reachable states using a FIFO queue. A visited set prevents repeated states and cycles. The search stops when

$$G \subseteq S.$$

If all reachable states are explored without satisfying the goal, the program prints **No plan found**.

5 Test Results

Test A: Solvable Problem

Using the original warehouse problem, the planner found:

$$PickUp(A) \rightarrow Move(A, B) \rightarrow Move(B, C) \rightarrow Drop(C)$$

The final state contains $At(Package, C)$.

Result: Plan found and valid.

Test B: Impossible Problem

All `PickUp` actions were removed. The robot could still move, but the package could never be carried to C .

The planner returned:

No plan found

Result: Correctly detected that no plan exists.

Test C: Irrelevant Actions

A direct robot movement from A to C was added. Reaching C with only the robot did not satisfy the goal because the required proposition is

$$At(Package, C),$$

not $At(Robot, C)$.

Result: Planner correctly distinguished robot location from package location.

6 Logic and Search

The planner works as:

Current State $\rightarrow$ Check Preconditions $\rightarrow$ Apply Valid Action $\rightarrow$ Generate Successor $\rightarrow$ Search $\rightarrow$ Goal Test

Logical reasoning decides whether

$$S \models Preconditions(a)$$

and determines the resulting state after an action.

BFS decides which valid state transitions to explore.

Therefore,

Logic determines what is possible; search determines what to try.

The independently executed state transitions should be trusted more than an LLM explanation because they directly verify the defined preconditions and effects.

7 Reflection Questions

1. Why specify preconditions and effects first?

They precisely define when actions are legal and how they change the state, reducing ambiguity in the generated planner.

2. **What error could occur without checking preconditions?**

The planner could drop the package at C even when the robot is not at C or is not holding it.

3. **Why is a reasonable-looking plan not necessarily valid?**

Every action must have its preconditions satisfied in the exact state where it is executed.

4. **What did the LLM contribute?**

It generated the Python structure, action representation, BFS planner, tests, and explanatory comments.

5. **What was verified independently?**

The program was executed and the returned plans, state transitions, goal conditions, and impossible case were checked.

6. **Where is logical reasoning used?**

It is used in checking action preconditions and applying positive and negative effects.

7. **How is planning related to search?**

States are search nodes, actions are transitions, the initial state is the start node, and goal-satisfying states are goal nodes.

8 LLM Assistance

ChatGPT was used to generate and explain the Python implementation. The generated program was then executed and tested independently using the three required cases.

Thus the workflow was

$Understand \rightarrow Specify \rightarrow Generate \rightarrow Execute \rightarrow Verify$
--